Section 42: Linear Algebra Fundamentals

Linear algebra is a foundational field of mathematics which focuses on vector spaces and linear *mappings* between them. It plays a central role in various areas of both pure and applied mathematics, featuring compact representations of linear systems through the use of matrices, efficient methods for solving problems featuring such systems, and the ability to quantify the behavior and stability of systems over time. Linear algebra has applications in many fields including circuit analysis, signal processing, control systems and control theory, communication, and power systems. Generally speaking, the utility of linear algebra in physics and electrical engineering cannot be overstated, as it enables the design, analysis, and optimization of a variety of complex systems. In this section, we will explore several elementary mathematical concepts and operations necessary to work with and understand linear systems.

Familiarization with software designed for numerical linear algebra and fast, numerically-robust matrix calculations will be indispensable moving forward. In particular, downloading and installing MATLAB is recommended, as well as completing the onramp tutorial course. GNU Octave is a compatible open-source alternative which can execute any MATLAB script which uses base MATLAB function libraries.

As an optional alternative to MATLAB or GNU Octave, standardized software libraries for numerical linear algebra can be used for popular high-level scientific programming languages such as Python or Julia. In the case of Python, the NumPy, SciPy, and Matplotlib libraries are very helpful for scientific programming and data visualization, for which installation and syntax documentation is provided through their respective hyperlinks. Through the linear algebra library package LAPACK, Julia provides native implementations of basic and common linear algebra operations with high-level syntax.

Section 42.1: Vectors

Vectors are fundamental to understanding concepts in physics and engineering, consisting of a magnitude and direction in an N-dimensional Euclidean space in which each dimension is orthogonal to all other dimensions in the $\mathbb{R}^N$ space. As such, they are useful for representing a variety of physical quantities such as electric fields, currents, or forces, for example.

Vector Notation and Algebra

Using Cartesian coordinates, a vector in a $N = 2$ dimensional space can be represented as $\mathbf{v} = [x\ y]$, and in a $N = 3$ dimensional space as $\mathbf{v} = [x\ y\ z]$, where x, y, and z are the components of the vector in

each dimension. As with numbers, the concept of operations carries over to vectors though with some modifications. Key operations include but are not limited to transposition, addition, subtraction, scalar multiplication, and the inner product.

Vectors can either be a $1 \times N$ row vector in which there is a single row of N elements (N columns),

$$\mathbf{v} = \begin{bmatrix} v_1 & v_2 & \cdots & v_N \end{bmatrix},$$

or an $N \times 1$ column vector in which there is a single column of N elements (N rows),

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \end{bmatrix}.$$

By convention, we will default to describing vectors as column vectors.[1] The transpose operation on vectors is a fundamental operation that involves converting a row vector into a column vector, or vice versa, denoted by the superscript $^\intercal$ on the vector symbol. If $\mathbf{v}$ is an $N \times 1$ column vector, its transpose $\mathbf{v}^\intercal$ is then a $1 \times N$ row vector. Conversely, if $\mathbf{v}$ is a $1 \times N$ row vector, its transpose $\mathbf{v}^\intercal$ is an $N \times 1$ column vector.[2] This is visually depicted in Fig. 1.

[1] By default, vectors are represented as $1 \times N$ row arrays in MATLAB.

[2] To convert a row array into a column array in MATLAB, the function transpose() is used. Thus, $\mathbf{v}^\intercal$ = transpose($\mathbf{v}$), where $\mathbf{v}$ is by default a $1 \times N$ row array and therefore $\mathbf{v}^\intercal$ is an $N \times 1$ column array.

Addition or subtraction of vectors requires that the dimensionality of all vectors being summed is equal. For example, adding or subtracting vectors $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ requires that they all either be row or column vectors with the same number of N elements:

$$\mathbf{a} + \mathbf{b} + \mathbf{c} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_N \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{bmatrix} + \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_N \end{bmatrix} = \begin{bmatrix} a_1 + b_1 + c_1 \\ a_2 + b_2 + c_2 \\ \vdots \\ a_N + b_N + c_N \end{bmatrix}.$$

The magnitude, or length, of an N-dimensional vector $\mathbf{v}$ is described by its *Euclidean* norm $\|\mathbf{v}\|_2$, also referred to as its 2-norm. It is calculated as

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + v_3^2 + \cdots + v_N^2}.$$

The formula may essentially be viewed as the Pythagorean theorem extended to N dimensions. The 2-norm gives a direct measure of the distance of the vector from the origin of the coordinate system, and has the properties of being non-negative as well as satisfying the *triangle inequality*, $\|\mathbf{u} + \mathbf{v}\|_2 \leq \|\mathbf{u}\|_2 + \|\mathbf{v}\|_2$, which asserts that the magnitude of the sum of two vectors is less than or equal to the sum of their individual magnitudes.

$$\mathbf{v}^\intercal = \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ \vdots \\ v_N \end{bmatrix}^\intercal = \begin{bmatrix} v_1 & v_2 & v_3 \cdots v_N \end{bmatrix}$$

Figure 1: The transpose of a vector may be viewed as rotating the array of numbers by 90° such that an $N \times 1$ column vector becomes a $1 \times N$ row vector. If this process is repeated on the row vector, the original column vector is recovered.

Vectors may also be multiplied by numbers, an operation known as *scalar* multiplication since the number *scales* the vector by stretching or shrinking its magnitude without changing its orientation, or direction, in the N-dimensional space. Thus, multiplying a scalar a with a vector $\mathbf{v}$ results in each of the vector elements scaled by a:

$$a\mathbf{v} = a \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \end{bmatrix} = \begin{bmatrix} av_1 \\ av_2 \\ \vdots \\ av_N \end{bmatrix}.$$

Finally, element-wise or *Hadamard* multiplication of two vectors results in a vector in which each element is a scalar product of the respective elements of the original two vectors:

$$\mathbf{u} \odot \mathbf{v} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{bmatrix} \odot \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_N \end{bmatrix} = \begin{bmatrix} u_1 v_1 \\ u_2 v_2 \\ \vdots \\ u_N v_N \end{bmatrix}.$$

Unit Vectors

A unit vector is a vector that has a magnitude of one. It is often used in engineering to represent direction, since it retains the directional properties of a vector without contributing to the magnitude. This makes unit vectors very useful for specifying directions in space and for normalizing vectors. As such, unit vectors are often used in linear algebra to define the axes of coordinate systems and can therefore be used as a geometrical basis. To calculate the unit vector of any given vector, each component of the vector is divided by the vector magnitude. The (unit-magnitude) vector $\hat{\mathbf{v}}$ in the direction of $\mathbf{v}$ is then given by

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|},$$

resulting in $\|\hat{\mathbf{v}}\| = 1$.

Unit vectors are widely used in vector calculations, including vector projections, cross and dot product operations, and when defining coordinate systems in both two and three dimensions. They serve as the foundational elements for much of vector calculus and physics involving vectors.

Inner Product and Hermitian Adjoint

If θ is the angle between two vectors, then the inner product may be defined as

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos(\theta) = ab \cos(\theta),$$

where $\|\mathbf{a}\|$ and $\|\mathbf{b}\|$ are the respective magnitudes of vectors $\mathbf{a}$ and $\mathbf{b}$. This procedure is visualized in Fig. 2, where the inner product between vectors $\mathbf{a}$ and $\mathbf{b}$ may be viewed as a projection of one vector onto the other. The inner product also satisfies the *Cauchy-Schwarz inequality*[3],

$$|\mathbf{a} \cdot \mathbf{b}| \leq \|\mathbf{a}\| \, \|\mathbf{b}\| \, ,$$

a bound which follows immediately from the geometric definition above, given that $|\cos(\theta)| \leq 1$.

In terms of the set of purely-real vector elements a_i and b_i with $i \in \{1, 2, \ldots, N\}$ for real vectors $\mathbf{a}$ and $\mathbf{b}$, respectively, the inner product is given as

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{N} a_i b_i .$$

The inner product can be used to determine whether two vectors are orthogonal. By definition, two vectors are orthogonal if their inner product is zero.

For the case in which the vector elements are complex, however, we must introduce additional algebra. By convention, a complex vector $\mathbf{u}$ is written as a column vector of complex values $u_i \in \mathbb{C}$:

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_N \end{bmatrix} .$$

We define the *Hermitian adjoint* of this vector by taking the complex conjugate *and* transpose of this vector:

$$\mathbf{u}^\dagger = (\mathbf{u}^*)^\intercal = (\mathbf{u}^\intercal)^* = \begin{bmatrix} u_1^* & u_2^* & \ldots & u_N^* \end{bmatrix} .$$

The inner product of two complex vectors $\mathbf{u}$ and $\mathbf{v}$ is then computed as

$$\mathbf{u}^\dagger \cdot \mathbf{v} = \sum_{i=1}^{N} u_i^* v_i .$$

Thus, we see that the inner product may be regarded as a *multiply-accumulate*, or MAC, operation. In addition, as in the case of purely-real vectors, two complex vectors are considered orthogonal if their inner product is zero. Finally, normalization of this vector requires that

$$\|\mathbf{v}\| = \sqrt{\mathbf{v}^\dagger \mathbf{v}} = \sqrt{\sum_{i=1}^{N} v_i^* v_i} = \sqrt{\sum_{i=1}^{N} |v_i|^2} = 1.$$

[3] The Cauchy-Schwarz inequality, also known as the Cauchy-Bunyakovsky-Schwarz inequality, is named after Augustin-Louis Cauchy, Viktor Bunyakovsky, and Hermann Schwarz. Augustin-Louis Cauchy first published the inequality for sums in 1821, while the corresponding inequality for integrals was published by Viktor Bunyakovsky in 1859 and Hermann Schwarz in 1888.

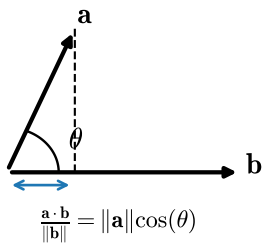

Figure 2: The inner product between vectors $\mathbf{a}$ and $\mathbf{b}$ involves the concept of projecting one vector onto another.

Exercises

Problem 1: Given a vector $\mathbf{v} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$, find the unit vector $\mathbf{u}$ in the direction of $\mathbf{v}$.

Problem 2: Calculate the inner product of vectors $\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}$ and $\mathbf{b} = \begin{bmatrix} -2 \\ 0 \\ 3 \end{bmatrix}$.

Problem 3: Consider two complex *orthogonal* vectors $\mathbf{v} = \begin{bmatrix} 3+4\mathrm{i} \\ a \end{bmatrix}$ and $\mathbf{u} = \frac{1}{2}\begin{bmatrix} 1+\mathrm{i} \\ b \end{bmatrix}$. Solve for all possible values of a and b, assuming that $\mathbf{u}$ is a unit vector and $b \in \mathbb{R}$.

Section 42.2: Matrices

Matrix Notation and Algebra

Matrices are rectangular arrays of numbers, symbols, or expressions arranged in rows and columns, widely used to solve systems of linear equations and to perform linear transformations. A matrix is typically denoted by a capital letter and can be represented as $\mathbf{A} = [a_{ij}]$, where i and j indicate the row and column positions of an element in the matrix. Common matrix operations include addition, subtraction, multiplication, and finding the determinant and inverse in the case of square matrices. Matrix multiplication is particularly useful for performing linear transformations and solving linear systems. As with numbers and vectors, matrices may be added, subtracted and scaled. However as with vectors, multiplication of matrices involves special rules. In this section, we provide a brief review of matrix arithmetic along with examples.

As in the case of vectors, we begin with the concept of transposing a matrix, which is a fundamental operation in linear algebra. Transposing a matrix involves rearranging the rows of a matrix into columns (or vice versa), which can be visually represented by flipping the matrix over its diagonal, as depicted in Fig. 3.

Given a matrix $\mathbf{A}$ of size $m \times n$, where m is the number of rows and n is the number of columns, the transpose of $\mathbf{A}$, denoted as $\mathbf{A}^\intercal$, is a new matrix of size $n \times m$. In particular, the elements of $\mathbf{A}^\intercal$ are defined such that the element at row i and column j in $\mathbf{A}^\intercal$ is the element at row j and column i in the original matrix $\mathbf{A}$. Thus, for $\mathbf{A} = [a_{ij}]$, we

$$\mathbf{A}^\intercal = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}^\intercal = \begin{bmatrix} a_{11} & a_{21} & a_{31} \\ a_{12} & a_{22} & a_{32} \\ a_{13} & a_{23} & a_{33} \end{bmatrix}$$

Figure 3: The transpose of a matrix may be viewed as swapping the upper and lower triangular submatrix elements by rotating the matrix about its diagonal, resulting in the swapping of off-diagonal matrix elements.

can define the transpose as

$$\mathbf{A}^\intercal = \left[a_{ji}\right].$$

There are several important and useful properties of transposed matrices. If $\mathbf{A}$ is a symmetric square matrix, then $\mathbf{A}^\intercal = \mathbf{A}$. In addition, the transpose of a transposed matrix is equal to the original matrix; that is, $(\mathbf{A}^\intercal)^\intercal = \mathbf{A}$. Furthermore, the transpose operation is distributive, meaning that the transpose of the sum of two matrices is equal to the sum of their transposes so that

$$(\mathbf{A} + \mathbf{B})^\intercal = \mathbf{A}^\intercal + \mathbf{B}^\intercal.$$

Finally, the transpose of a product of matrices is equal to the product of their transposes in reverse order; that is,

$$(\mathbf{AB})^\intercal = \mathbf{B}^\intercal\mathbf{A}^\intercal.$$

Addition of matrices is commutative, that is, for two matrices $\mathbf{A}$ and $\mathbf{B}$, $\mathbf{A}$+$\mathbf{B}$=$\mathbf{B}$+$\mathbf{A}$. Whether two matrices are added or subtracted, however, they must have the same dimensions. That is, the number of rows m and columns n must match. Thus as an example, for a set of 2×2 matrices $\mathbf{A}$ and $\mathbf{B}$,

$$\mathbf{A} \pm \mathbf{B} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \pm \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11} \pm b_{11} & a_{12} \pm b_{12} \\ a_{21} \pm b_{21} & a_{22} \pm b_{22} \end{bmatrix}.$$

One type of multiplication of matrices is known as the Hadamard, or element-wise, product, which is also defined for two matrices of identical dimensions. As with multiplication of numbers, the Hadamard product is commutative. Using the 2×2 matrices above, the Hadamard product would be computed as follows,

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \odot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} \\ a_{21}b_{21} & a_{22}b_{22} \end{bmatrix}.$$

This element-wise operation can be quite useful when performing numerical operations on large arrays of numbers.

The more common type of matrix product is a multiply-accumulate (MAC) operation which is *not* commutative and requires that the number of columns of the left matrix match the number of rows of the right matrix. In general, for an $m \times n$ dimensional matrix $\mathbf{A}$ and an $n \times p$ dimensional matrix $\mathbf{B}$, the matrix $\mathbf{C}$ resulting from the product $\mathbf{C} = \mathbf{AB}$ will have dimension $m \times p$. As an example, consider a 2×2 matrix $\mathbf{A}$ and a 2×2 matrix $\mathbf{B}$. The matrix product $\mathbf{AB}$ is then calculated by taking the first row of $\mathbf{A}$ and multiplying the various column elements with the respective set of row elements of the first column of $\mathbf{B}$. These

products are then added to obtain the first row, first column element value of the resulting product matrix. Thus, we have

$$\mathbf{AB} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} (a_{11}b_{11} + a_{12}b_{21}) & (a_{11}b_{12} + a_{12}b_{22}) \\ (a_{21}b_{11} + a_{22}b_{21}) & (a_{21}b_{12} + a_{22}b_{22}) \end{bmatrix}.$$

Matrix Determinants

For a 2×2 matrix $\mathbf{A}$, the determinant is given by

$$\det(\mathbf{A}) \equiv |\mathbf{A}| = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}.$$

For a 3×3 matrix $\mathbf{B}$, the determinant is given by

$$\det(\mathbf{B}) \equiv |\mathbf{B}| = \begin{vmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{vmatrix}$$
$$= b_{11}(b_{22}b_{33} - b_{23}b_{32}) - b_{12}(b_{21}b_{33} - b_{23}b_{31}) + b_{13}(b_{21}b_{32} - b_{22}b_{31}),$$

where three determinants are computed from 2×2 submatrices which do not share the row and column indices of each element in the first row. This procedure can be generalized to larger $n \times n$ matrices, in which the sign associated with the element in column j of the first row alternates as $(-1)^{1+j}$.

Outer Product

A matrix can also be formed from two vectors by computing the *outer*

product. As opposed to an inner product in which a row vector is multiplied with a column vector, the outer product is performed by multiplying a column vector with a row vector, resulting in a matrix. For an $m \times 1$ complex vector $\mathbf{u}$ and an $n \times 1$ complex vector $\mathbf{v}$ where

$$\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix} \text{ and } \mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$$

the outer product is defined as:

$$\mathbf{u}\mathbf{v}^\dagger = \mathbf{u}\,(\mathbf{v}^*)^\intercal = \mathbf{u}\,(\mathbf{v}^\intercal)^* = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix} \begin{bmatrix} v_1^* & v_2^* & \dots & v_n^* \end{bmatrix}$$
$$= \begin{bmatrix} u_1 v_1^* & u_1 v_2^* & \cdots & u_1 v_n^* \\ u_2 v_1^* & u_2 v_2^* & \cdots & u_2 v_n^* \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1^* & u_m v_2^* & \dots & u_m v_n^* \end{bmatrix}.$$

Thus, we see that the resulting matrix has dimensions $m \times n$.

Tensor Product

Tensor products, also known as Kronecker products, are mathematical operations that extend the concept of an outer product of vectors to matrices. Such an operation is widely used in quantum computing as well as signal processing. The tensor product combines the dimensionality of two vectors to create a higher dimensional space.

If column vector $\mathbf{a}$ has N_1 elements $(a_1, \dots, a_{N_1})$ and column vector $\mathbf{b}$ has N_2 elements $(b_1, \dots, b_{N_2})$, the *product state* $\mathbf{a} \otimes \mathbf{b}$ is *also* a column vector of length $N_1 N_2$,

$$\mathbf{a} \otimes \mathbf{b} = \begin{bmatrix} a_1 \mathbf{b} \\ a_2 \mathbf{b} \\ \vdots \\ a_{N_1} \mathbf{b} \end{bmatrix} = \begin{bmatrix} a_1 b_1 \\ a_1 b_2 \\ \vdots \\ a_1 b_{N_2} \\ a_2 b_1 \\ a_2 b_2 \\ \vdots \\ a_{N_1} b_{N_2} \end{bmatrix}$$

For example,

$$\begin{bmatrix} a_1 \\ a_2 \end{bmatrix} \otimes \begin{bmatrix} b_1 \\ b_2 \end{bmatrix} = \begin{bmatrix} a_1 b_1 \\ a_1 b_2 \\ a_2 b_1 \\ a_2 b_2 \end{bmatrix}$$

More generally, if matrix $\mathbf{A} = [a_{ij}]$ has dimensions $m \times n$ and matrix $\mathbf{B} = [b_{ij}]$ has dimensions $p \times q$, then the tensor product is the $mp \times nq$ block matrix

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11}\mathbf{B} & \dots & a_{1n}\mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\mathbf{B} & \dots & a_{mn}\mathbf{B} \end{bmatrix}$$

For example,

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \otimes \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}\mathbf{B} & a_{12}\mathbf{B} \\ a_{21}\mathbf{B} & a_{22}\mathbf{B} \end{bmatrix}$$
$$= \begin{bmatrix} a_{11}b_{11} & a_{11}b_{12} & a_{12}b_{11} & a_{12}b_{12} \\ a_{11}b_{21} & a_{11}b_{22} & a_{12}b_{21} & a_{12}b_{22} \\ a_{21}b_{11} & a_{21}b_{12} & a_{22}b_{11} & a_{22}b_{12} \\ a_{21}b_{21} & a_{21}b_{22} & a_{22}b_{21} & a_{22}b_{22} \end{bmatrix}$$

Inverse of a Matrix

Understanding how to find the inverse of a matrix is a fundamental concept in linear algebra, with extensive applications in engineering. While the inverse of a matrix is not guaranteed to exist, it can be a powerful tool used for solving systems of linear equations, analyzing electrical circuits, optimization of a variety of systems, and more. The inverse of a square matrix $\mathbf{A}$ is another square matrix, denoted as $\mathbf{A}^{-1}$, such that when $\mathbf{A}$ is multiplied by $\mathbf{A}^{-1}$, the result is the identity matrix $\mathbf{I}$, such that

$$\mathbf{A}^{-1}\mathbf{A} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}.$$

The identity matrix is defined for any size $n \times n$ and consists of ones on the diagonal and zeros elsewhere. Its general form is given as

$$\mathbf{I} = \begin{bmatrix} 1 & 0 & \cdots & \cdots & 0 \\ 0 & 1 & & & \vdots \\ \vdots & & \ddots & & \vdots \\ \vdots & & & 1 & 0 \\ 0 & \cdots & \cdots & 0 & 1 \end{bmatrix}.$$

Importantly, the identity matrix serves as the multiplicative identity in matrix multiplication. For any $n \times n$ matrix $\mathbf{A}$,

$$\mathbf{I}\mathbf{A} = \mathbf{A}\mathbf{I} = \mathbf{A}.$$

Thus, multiplying any matrix by the identity matrix leaves the original matrix unchanged. Two conditions required for a matrix to have an inverse are that it must be square and the determinant of the matrix must be non-zero. If the determinant is zero, the matrix is referred to as *singular* with no defined inverse. There are several methods which may be used to compute the inverse of a matrix, including Gaussian elimination and analytic methods. Since manually computing the inverse of an invertible matrix quickly becomes impractical with respect to the matrix size, it is best to either automate the procedure with a computer program or use an optimized library function in a language

such as MATLAB or Python. In MATLAB, for example, the inverse of a matrix may be computed using the inverse function, inv(), such that $\mathbf{A}^{-1} = \text{inv}(\mathbf{A})$.

Gaussian Elimination

Gaussian elimination is a systematic method for solving systems of linear equations, and it can also be used to find the inverse of a matrix. The process involves augmenting the original matrix $\mathbf{A}$ with the identity matrix $\mathbf{I}$ and then performing row operations to transform $\mathbf{A}$ into $\mathbf{I}$. Allowed row operations include swapping rows, multiplying a row by a nonzero scalar, and adding a multiple of one row to another, with the goal of transforming the original matrix into an *upper triangular* form and then into *reduced row echelon* form. The resulting matrix on the right-hand side of the augmented matrix will then be $\mathbf{A}^{-1}$.

For Gaussian elimination to be used to find the inverse of a matrix, the matrix must be square (same number of rows and columns) and invertible, meaning that its determinant must be non-zero. If the determinant is zero, the matrix is *singular* and does not have an inverse. This singularity occurs if the matrix has linearly-dependent rows or columns, leading to at least one entire row of zeros in its row-reduced form.

Example: Consider the matrix

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & -1 \\ 1 & -1 & 1 \\ -1 & 1 & 1 \end{bmatrix}.$$

We begin by augmenting it with the identity matrix on the right, then adding the first row to the third row, followed by subtracting the first row from the second row:

$$[\,\mathbf{A} \mid \mathbf{I}\,] = \left[\begin{array}{ccc|ccc} 1 & 1 & -1 & 1 & 0 & 0 \\ 1 & -1 & 1 & 0 & 1 & 0 \\ -1 & 1 & 1 & 0 & 0 & 1 \end{array}\right]$$

$$\Rightarrow \left[\begin{array}{ccc|ccc} 1 & 1 & -1 & 1 & 0 & 0 \\ 1 & -1 & 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 & 0 & 1 \end{array}\right] \Rightarrow \left[\begin{array}{ccc|ccc} 1 & 1 & -1 & 1 & 0 & 0 \\ 0 & -2 & 2 & -1 & 1 & 0 \\ 0 & 2 & 0 & 1 & 0 & 1 \end{array}\right].$$

For the next several linear operations towards transforming the left-hand side of the augmented matrix into the identity matrix, we multiply the second row by $-1/2$, then subtract the second row from the first row, followed by subtracting 2 times the second row from the third row:

$$\Rightarrow \left[\begin{array}{ccc|ccc} 1 & 1 & -1 & 1 & 0 & 0 \\ 0 & 1 & -1 & \frac{1}{2} & -\frac{1}{2} & 0 \\ 0 & 2 & 0 & 1 & 0 & 1 \end{array}\right] \Rightarrow \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 1 & -1 & \frac{1}{2} & -\frac{1}{2} & 0 \\ 0 & 2 & 0 & 1 & 0 & 1 \end{array}\right]$$
$$\Rightarrow \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 1 & -1 & \frac{1}{2} & -\frac{1}{2} & 0 \\ 0 & 0 & 2 & 0 & 1 & 1 \end{array}\right].$$

Finally, we multiply the third row by $1/2$ and then add the third row to the second row, resulting in the identity matrix on the left-hand side of the augmented matrix and the inverse $\mathbf{A}^{-1}$ of the original matrix $\mathbf{A}$ on the right-hand side:

$$\Rightarrow \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 1 & -1 & \frac{1}{2} & -\frac{1}{2} & 0 \\ 0 & 0 & 1 & 0 & \frac{1}{2} & \frac{1}{2} \end{array}\right] \Rightarrow \left[\begin{array}{ccc|ccc} 1 & 0 & 0 & \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & 1 & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 0 & 1 & 0 & \frac{1}{2} & \frac{1}{2} \end{array}\right]$$
$$= \left[\begin{array}{c|c} \mathbf{I} & \mathbf{A}^{-1} \end{array}\right].$$

Thus, the matrix inverse is $\mathbf{A}^{-1} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & \frac{1}{2} \end{bmatrix}$.

Analytic solution to matrix inversion

An alternative method for computing the inverse of a matrix is to do so analytically with the cofactor matrix method. Also known as the adjugate method, it is based on computing the cofactor matrix of a square matrix $\mathbf{A}$, transposing it to get the adjugate matrix, and finally dividing each element by the determinant of $\mathbf{A}$. This method can be expressed by first defining the *cofactor* matrix.

Given a matrix $\mathbf{A} = [a_{ij}]$, the cofactor c_{ij} is given by multiplying the *minor* determinant associated with element a_{ij} with $(-1)^{i+j}$, in which the minor matrix is the submatrix comprised of matrix elements which do not share the row *nor* column number of matrix element a_{ij}.

Example 1: Assuming we have the 3×3 matrix

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix},$$

if we wish to compute the cofactor associated with, say, element a_{21}, we first identify the minor matrix $M_{2,1}$ corresponding to this element and compute the determinant:

$$M_{2,1} = \begin{vmatrix} \cancel{a_{11}} & a_{12} & a_{13} \\ \cancel{a_{21}} & \cancel{a_{22}} & \cancel{a_{23}} \\ \cancel{a_{31}} & a_{32} & a_{33} \end{vmatrix} = \begin{vmatrix} a_{12} & a_{13} \\ a_{32} & a_{33} \end{vmatrix} = a_{12}a_{33} - a_{13}a_{32}$$

The cofactor c_{21} is then found by multiplying this determinant with $(-1)^{i+j}$, where $i = 2$ and $j = 1$:

$$c_{21} = (-1)^{2+1} M_{2,1} = -(a_{12}a_{33} - a_{13}a_{32}) = a_{13}a_{32} - a_{12}a_{33}.$$

Thus, if matrix $\mathbf{A}$ is invertible, the inverse $\mathbf{A}^{-1}$ is given in terms of the transpose of the cofactor matrix $\mathbf{C} = [c_{ij}]$ and the determinant $|\mathbf{A}|$ as

$$\mathbf{A}^{-1} = \frac{\mathbf{C}^{\mathsf{T}}}{|\mathbf{A}|}.$$

Example: Compute the inverse of a general 2×2 matrix:
Given a 2×2 matrix $\mathbf{A}$,

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix},$$

the inverse is straightforwardly computed as

$$\begin{aligned} \mathbf{A}^{-1} &= \frac{1}{(a_{11}a_{22} - a_{12}a_{21})} \begin{bmatrix} a_{22} & -a_{21} \\ -a_{12} & a_{11} \end{bmatrix}^{\mathsf{T}} \\ &= \frac{1}{(a_{11}a_{22} - a_{12}a_{21})} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix}. \end{aligned}$$

Solving Systems of Linear Equations

One of the most common applications of the matrix inverse in electrical engineering is solving a system of n linear equations in n unknowns, written compactly in matrix-vector form as

$$\mathbf{A}\mathbf{x} = \mathbf{b},$$

where $\mathbf{A}$ is an $n \times n$ coefficient matrix, $\mathbf{x}$ is the column vector of un-

knowns, and $\mathbf{b}$ is the column vector of known source terms. If $\mathbf{A}$ is invertible, the unique solution is obtained by left-multiplying both sides by $\mathbf{A}^{-1}$:

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}.$$

If instead $\mathbf{A}$ is singular, the system either has no solution or infinitely many solutions, since at least one equation is a linear combination of the others.[4]

[4] In circuit terms, a singular conductance matrix typically indicates a modeling error, such as a floating node or a redundant constraint.

An equivalent analytic approach for small systems is *Cramer's rule*, in which each unknown is expressed as a ratio of determinants,

$$x_i = \frac{|\mathbf{A}_i|}{|\mathbf{A}|},$$

where $\mathbf{A}_i$ is the matrix $\mathbf{A}$ with its i-th column replaced by $\mathbf{b}$. In numerical practice, neither the explicit inverse nor Cramer's rule is used for large systems; optimized routines based on Gaussian elimination are preferred.[5] This formulation is precisely what arises in nodal analysis of circuits, where Kirchhoff's current law at each node produces one linear equation, as illustrated in the following example.

[5] In MATLAB, the backslash (left-division) operator, $\mathbf{x} = \mathbf{A}\backslash\mathbf{b}$, solves the system directly via optimized elimination routines and is both faster and more numerically robust than computing $\mathrm{inv}(\mathbf{A}) * \mathbf{b}$.

Example: Nodal analysis as a linear system

Consider a resistive circuit with two node voltages v_1 and v_2: a current source $I_s = 5$ mA drives node 1, resistor $R_1 = 1$ kΩ connects node 1 to ground, $R_2 = 1$ kΩ connects node 1 to node 2, and $R_3 = 1$ kΩ connects node 2 to ground. Writing Kirchhoff's current law at each node in terms of the conductances $G_i = 1/R_i = 1$ mS yields the system

$$\begin{bmatrix} G_1 + G_2 & -G_2 \\ -G_2 & G_2 + G_3 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} I_s \\ 0 \end{bmatrix}$$
$$\Rightarrow\ 10^{-3} \begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 5 \times 10^{-3} \\ 0 \end{bmatrix}.$$

Using the analytic 2×2 inverse derived previously, with determinant $|\mathbf{G}| = (4-1) \times 10^{-6} = 3 \times 10^{-6}$,

$$\begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \frac{10^{-3}}{3 \times 10^{-6}} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 5 \times 10^{-3} \\ 0 \end{bmatrix} = \frac{10^3}{3} \begin{bmatrix} 10 \times 10^{-3} \\ 5 \times 10^{-3} \end{bmatrix} = \begin{bmatrix} 10/3 \\ 5/3 \end{bmatrix} \text{ V},$$

so that $v_1 \approx 3.33$ V and $v_2 \approx 1.67$ V. As a sanity check, the source delivers 5 mA, of which $v_1/R_1 = 3.33$ mA flows through R_1 and $(v_1 - v_2)/R_2 = 1.67$ mA flows toward node 2, satisfying KCL at node 1.

Eigenvectors and Eigenvalues

For a square $n \times n$ matrix $\mathbf{A}$, an *eigenvector* is a nonzero vector $\mathbf{v}$ whose direction is left unchanged by multiplication with $\mathbf{A}$; the matrix merely scales the vector by a factor λ, called the corresponding *eigenvalue*. This defining property is expressed by the eigenvalue equation

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}, \qquad \mathbf{v} \neq \mathbf{0}.$$

Rearranging gives $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = \mathbf{0}$, which can only have a nonzero solution $\mathbf{v}$ if the matrix $\mathbf{A} - \lambda\mathbf{I}$ is singular. The eigenvalues are therefore found as the roots of the *characteristic equation*

$$\det(\mathbf{A} - \lambda\mathbf{I}) = 0,$$

which is a polynomial of degree n in λ, yielding exactly n eigenvalues when counted with multiplicity (some of which may be repeated or complex). Once each eigenvalue λ_i is known, the associated eigenvector is obtained by solving the singular system $(\mathbf{A} - \lambda_i\mathbf{I})\mathbf{v}_i = \mathbf{0}$. Since any scalar multiple of an eigenvector is also an eigenvector, eigenvectors are conventionally normalized to unit magnitude.[6]

[6] In MATLAB, $[\mathbf{V}, \mathbf{D}] = \mathrm{eig}(\mathbf{A})$ returns a matrix $\mathbf{V}$ whose columns are the (unit-normalized) eigenvectors and a diagonal matrix $\mathbf{D}$ of the corresponding eigenvalues.

Several useful properties follow directly from the characteristic polynomial. The sum of the eigenvalues equals the *trace* of the matrix (the sum of its diagonal elements), while the product of the eigenvalues equals the determinant:

$$\sum_{i=1}^{n} \lambda_i = \mathrm{tr}(\mathbf{A}), \qquad \prod_{i=1}^{n} \lambda_i = \det(\mathbf{A}).$$

Consequently, a matrix is singular if and only if at least one eigenvalue is zero. In addition, the eigenvalues of a triangular (or diagonal) matrix are simply its diagonal entries, and a real *symmetric* matrix always has purely real eigenvalues with mutually orthogonal eigenvectors — a fact that will be essential for the principal component analysis discussed in Section 42.3.

Eigenvalues are of central importance in electrical engineering because they characterize the *natural response* of linear dynamic systems.[7] If a matrix has n linearly independent eigenvectors, collecting them as the columns of a matrix $\mathbf{P}$ allows the matrix to be *diagonalized* as

$$\mathbf{A} = \mathbf{P}\mathbf{D}\mathbf{P}^{-1},$$

where $\mathbf{D}$ is the diagonal matrix of eigenvalues. This factorization decouples systems of differential equations into independent first-order modes and makes matrix powers trivial to compute, since $\mathbf{A}^k = \mathbf{P}\mathbf{D}^k\mathbf{P}^{-1}$

[7] For a linear state-space system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$, each eigenpair contributes a natural mode $\mathbf{v}_i e^{\lambda_i t}$ to the solution. The eigenvalues coincide with the system poles: negative real parts imply decaying (stable) modes, while complex-conjugate pairs produce oscillatory modes, as in an RLC circuit.

with $\mathbf{D}^k$ obtained by simply raising each diagonal entry to the k-th power.

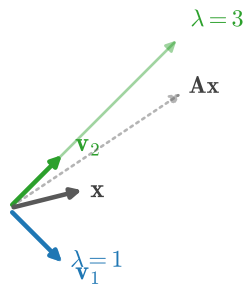

Figure 4: The action of $\mathbf{A} = \left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$ on its eigenvectors: each eigenvector is stretched by its eigenvalue ($\lambda_2 = 3$ along $\mathbf{v}_2$, $\lambda_1 = 1$ along $\mathbf{v}_1$) without any change in direction, while a generic vector is both rotated and scaled.

Example 1: Eigenvalues of a real symmetric matrix
Consider the symmetric matrix

$$\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}.$$

The characteristic equation is

$$\begin{aligned} \det(\mathbf{A} - \lambda \mathbf{I}) &= \begin{vmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{vmatrix} = (2-\lambda)^2 - 1 \\ &= \lambda^2 - 4\lambda + 3 = (\lambda - 1)(\lambda - 3) = 0, \end{aligned}$$

yielding eigenvalues $\lambda_1 = 1$ and $\lambda_2 = 3$. For $\lambda_1 = 1$, solving $(\mathbf{A} - \mathbf{I})\mathbf{v}_1 = \mathbf{0}$ gives

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \;\Rightarrow\; v_x = -v_y \;\Rightarrow\; \mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix},$$

while for $\lambda_2 = 3$, solving $(\mathbf{A} - 3\mathbf{I})\mathbf{v}_2 = \mathbf{0}$ gives $v_x = v_y$, so that $\mathbf{v}_2 = \frac{1}{\sqrt{2}}\left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$. Direct multiplication verifies the results, e.g., $\mathbf{A}\mathbf{v}_2 = \frac{1}{\sqrt{2}}\left[\begin{smallmatrix} 3 \\ 3 \end{smallmatrix}\right] = 3\mathbf{v}_2$. As expected for a symmetric matrix, the eigenvalues are real and the eigenvectors are orthogonal, $\mathbf{v}_1 \cdot \mathbf{v}_2 = 0$. Finally, the trace and determinant checks are satisfied: $\lambda_1 + \lambda_2 = 4 = \operatorname{tr}(\mathbf{A})$ and $\lambda_1 \lambda_2 = 3 = \det(\mathbf{A})$.

Example 2: Complex eigenvalues and oscillation
Real matrices need not have real eigenvalues. Consider

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix},$$

whose characteristic equation is $\lambda^2 + 1 = 0$, giving the purely imaginary conjugate pair $\lambda_{1,2} = \pm \mathrm{i}$. For $\lambda_1 = \mathrm{i}$, the condition $(\mathbf{A} - \mathrm{i}\mathbf{I})\mathbf{v} = \mathbf{0}$ requires $-\mathrm{i}v_x + v_y = 0$, so

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\ \mathrm{i}\end{bmatrix}, \qquad \mathbf{v}_2 = \mathbf{v}_1^* = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\ -\mathrm{i}\end{bmatrix}.$$

Complex eigenvalues of a real matrix always occur in conjugate pairs and indicate rotation rather than pure stretching: this particular matrix rotates every vector by 90°, so no real direction can be left unchanged. In the context of the dynamic system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$, the purely imaginary eigenvalues $\lambda = \pm\mathrm{i}$ correspond to sustained oscillation at unit angular frequency — precisely the behavior of an ideal lossless LC tank, whose state (capacitor voltage and inductor current) circulates indefinitely without decay.

Exercises

Problem 1: Given the matrices $\mathbf{A} = \begin{bmatrix}1 & 2\\ 3 & 4\end{bmatrix}$ and $\mathbf{B} = \begin{bmatrix}0 & 1\\ 1 & 0\end{bmatrix}$, compute both matrix products $\mathbf{AB}$ and $\mathbf{BA}$, and thereby confirm that matrix multiplication is not commutative. In addition, compute the Hadamard product $\mathbf{A} \odot \mathbf{B}$ and the tensor product $\mathbf{A} \otimes \mathbf{B}$.

Problem 2: Compute the determinant and the inverse of the matrix $\mathbf{A} = \begin{bmatrix}2 & 1\\ 5 & 3\end{bmatrix}$, and verify that $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$.

Problem 3: Solve the linear system $\begin{bmatrix}3 & -1\\ -1 & 2\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix} = \begin{bmatrix}5\\ 0\end{bmatrix}$ twice: once using the analytic 2×2 matrix inverse, and once using Cramer's rule. Confirm that both methods agree.

Problem 4: Find the eigenvalues and unit-normalized eigenvectors of $\mathbf{A} = \begin{bmatrix}4 & 1\\ 2 & 3\end{bmatrix}$. Verify your eigenvalues by checking them against the trace and determinant of $\mathbf{A}$.

Section 42.3: Advanced Topics

The topics in this section build directly on the eigenvalue machinery developed above and appear throughout modern electrical engineering practice — in signal processing, communications, data analysis, and machine learning. A first exposure at this stage is intended to establish vocabulary and geometric intuition; each topic is treated in far greater depth in later coursework.

Singular Value Decomposition (SVD)

While the eigenvalue decomposition applies only to square matrices (and diagonalization is not even guaranteed for all of those), the *sin-*

gular value decomposition exists for *every* matrix, square or rectangular, real or complex. Any $m \times n$ matrix $\mathbf{A}$ can be factored as

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\dagger},$$

where $\mathbf{U}$ is an $m \times m$ unitary matrix whose columns $\mathbf{u}_i$ are the *left singular vectors*, $\mathbf{V}$ is an $n \times n$ unitary matrix whose columns $\mathbf{v}_i$ are the *right singular vectors*, and $\boldsymbol{\Sigma}$ is an $m \times n$ diagonal matrix containing the real, non-negative *singular values* sorted in descending order, $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.[8]

The SVD is intimately connected to the eigenvalue problem: since $\mathbf{A}^{\dagger}\mathbf{A} = \mathbf{V}\boldsymbol{\Sigma}^{\mathsf{T}}\boldsymbol{\Sigma}\mathbf{V}^{\dagger}$, the right singular vectors are the eigenvectors of the square matrix $\mathbf{A}^{\dagger}\mathbf{A}$, and the singular values are the square roots of its (always non-negative) eigenvalues, $\sigma_i = \sqrt{\lambda_i}$. The left singular vectors then follow from the relation $\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i$. Geometrically, any matrix maps the unit circle (or hypersphere) into an ellipse (or hyperellipsoid) whose principal semi-axes have lengths σ_i and directions $\mathbf{u}_i$, as depicted in Fig. 5.

The singular values quantify how strongly a matrix amplifies inputs along its principal directions, and several important matrix properties follow immediately. The *rank* of a matrix equals the number of nonzero singular values. The ratio $\sigma_{\max}/\sigma_{\min}$, called the *condition number*, measures how numerically sensitive the solution of $\mathbf{A}\mathbf{x} = \mathbf{b}$ is to small perturbations. Furthermore, expanding the factorization as a sum of rank-one outer products,

$$\mathbf{A} = \sum_i \sigma_i \mathbf{u}_i \mathbf{v}_i^{\dagger},$$

and truncating the sum after the k largest singular values yields the best rank-k approximation of $\mathbf{A}$ — the principle underlying SVD-based image compression, noise reduction, and the decomposition of MIMO wireless channels into independent parallel data streams.

[8] For purely real matrices, the Hermitian adjoints reduce to transposes and the factorization reads $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\mathsf{T}}$. In MATLAB, the decomposition is computed with $[\mathbf{U}, \boldsymbol{\Sigma}, \mathbf{V}] = \mathrm{svd}(\mathbf{A})$.

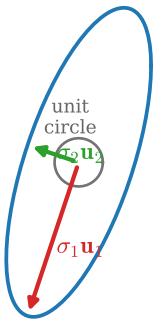

Figure 5: Any matrix maps the unit circle into an ellipse. For $\mathbf{A} = \left[\begin{smallmatrix} 3 & 0 \\ 4 & 5 \end{smallmatrix}\right]$, the semi-axes of the resulting ellipse have lengths equal to the singular values $\sigma_1 = 3\sqrt{5}$ and $\sigma_2 = \sqrt{5}$, oriented along the left singular vectors $\mathbf{u}_1$ and $\mathbf{u}_2$.

Example: SVD of a 2×2 matrix
Compute the singular value decomposition of

$$\mathbf{A} = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}.$$

First, form the symmetric matrix

$$\mathbf{A}^{\mathsf{T}}\mathbf{A} = \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix}\begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}.$$

Its characteristic equation $(25-\lambda)^2 - 400 = 0$ gives $\lambda_1 = 45$ and

$\lambda_2 = 5$, so the singular values are

$$\sigma_1 = \sqrt{45} = 3\sqrt{5}, \qquad \sigma_2 = \sqrt{5}.$$

The corresponding unit eigenvectors of $\mathbf{A}^\mathsf{T}\mathbf{A}$ (the right singular vectors) are found as in the previous examples:

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}, \qquad \mathbf{v}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix}.$$

The left singular vectors then follow from $\mathbf{u}_i = \mathbf{A}\mathbf{v}_i/\sigma_i$:

$$\mathbf{u}_1 = \frac{1}{3\sqrt{5}}\cdot\frac{1}{\sqrt{2}}\begin{bmatrix}3\\9\end{bmatrix} = \frac{1}{\sqrt{10}}\begin{bmatrix}1\\3\end{bmatrix}, \qquad \mathbf{u}_2 = \frac{1}{\sqrt{5}}\cdot\frac{1}{\sqrt{2}}\begin{bmatrix}3\\-1\end{bmatrix} = \frac{1}{\sqrt{10}}\begin{bmatrix}3\\-1\end{bmatrix}.$$

Note that $\mathbf{u}_1 \cdot \mathbf{u}_2 = 0$, as required for a unitary $\mathbf{U}$. Assembling the factors,

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\mathsf{T} = \frac{1}{\sqrt{10}}\begin{bmatrix}1 & 3\\3 & -1\end{bmatrix}\begin{bmatrix}3\sqrt{5} & 0\\0 & \sqrt{5}\end{bmatrix}\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix},$$

which may be verified by direct multiplication. As a further check, $\sigma_1\sigma_2 = 15 = |\det(\mathbf{A})|$.

Principal Component Analysis (PCA)

Principal component analysis is a widely-used statistical technique, built directly on the eigenvalue decomposition, for identifying the directions along which a set of measured data varies the most. Suppose N measurements are collected, each consisting of n variables (for example, simultaneous voltage readings from n sensors), and let $\mathbf{d}_i$ denote the i-th measurement vector after subtracting the mean of the dataset, $\boldsymbol{\mu} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{x}_i$. The sample *covariance matrix* is then constructed as a normalized sum of outer products of the mean-centered data,

$$\mathbf{C} = \frac{1}{N-1}\sum_{i=1}^{N}\mathbf{d}_i\mathbf{d}_i^\mathsf{T},$$

resulting in a real, symmetric $n \times n$ matrix whose diagonal entries are the variances of each variable and whose off-diagonal entries measure the correlations between pairs of variables. Because $\mathbf{C}$ is symmetric, its eigenvalues are real and its eigenvectors orthogonal. The unit eigenvectors of $\mathbf{C}$, called the *principal components*, define a rotated coordinate system aligned with the data: the first principal compo-

nent points along the direction of maximum variance, and each eigenvalue λ_i equals the variance of the data along its component. The fraction of the total variance captured by the first k components is $\sum_{i=1}^{k}\lambda_i / \sum_{i=1}^{n}\lambda_i$; when a few components capture most of the variance, the data can be *projected* onto those components with little loss of information, reducing dimensionality and suppressing measurement noise.[9]

[9] PCA is equivalent to computing the SVD of the mean-centered data matrix: the right singular vectors are the principal components, and the squared singular values are proportional to the variances λ_i. Numerical implementations exploit this equivalence, e.g., pca() in the MATLAB Statistics and Machine Learning Toolbox.

Example: PCA of a small dataset

Consider four two-dimensional measurements: $(5,4)$, $(1,2)$, $(4,5)$, and $(2,1)$. The mean is $\boldsymbol{\mu} = (3,3)$, so the mean-centered deviation vectors are

$$\mathbf{d}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{d}_2 = \begin{bmatrix} -2 \\ -1 \end{bmatrix}, \quad \mathbf{d}_3 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{d}_4 = \begin{bmatrix} -1 \\ -2 \end{bmatrix}.$$

With $N-1 = 3$, the covariance matrix is

$$\mathbf{C} = \frac{1}{3}\sum_{i=1}^{4}\mathbf{d}_i\mathbf{d}_i^{\mathsf{T}} = \frac{1}{3}\begin{bmatrix} 10 & 8 \\ 8 & 10 \end{bmatrix}.$$

By the symmetry of this matrix (compare Example 1 of the eigenvalue discussion), the unit eigenvectors are $\mathbf{w}_1 = \frac{1}{\sqrt{2}}\left[\begin{smallmatrix} 1 \\ 1 \end{smallmatrix}\right]$ and $\mathbf{w}_2 = \frac{1}{\sqrt{2}}\left[\begin{smallmatrix} 1 \\ -1 \end{smallmatrix}\right]$, with eigenvalues

$$\lambda_1 = \frac{10+8}{3} = 6, \qquad \lambda_2 = \frac{10-8}{3} = \frac{2}{3}.$$

The first principal component $\mathbf{w}_1$ therefore captures

$$\frac{\lambda_1}{\lambda_1+\lambda_2} = \frac{6}{6+2/3} = 90\%$$

of the total variance: the data are essentially one-dimensional along the diagonal direction, and projecting each measurement onto $\mathbf{w}_1$ (a single number per measurement instead of two) preserves nearly all of the information.

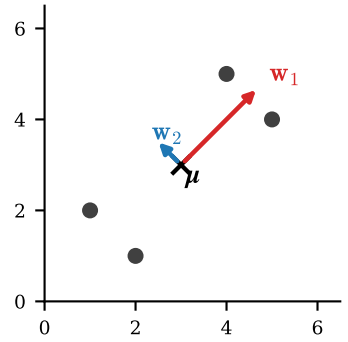

Figure 6: PCA of the four-point dataset in the worked example. The principal components $\mathbf{w}_1$ and $\mathbf{w}_2$ (drawn from the data mean and scaled by $\sqrt{\lambda_i}$) form an orthogonal coordinate system aligned with the direction of maximum variance.

Convolution Matrices

Discrete convolution is the fundamental operation performed by every linear time-invariant (LTI) system, including all digital FIR filters: the output signal is the convolution of the input signal with the system's impulse response,

$$y[n] = (h * x)[n] = \sum_k h[k]\, x[n-k].$$

Since each output sample is a weighted sum of input samples — a multiply-accumulate operation — convolution is a *linear* operation and can therefore be written as a matrix-vector product,

$$\mathbf{y} = \mathbf{H}\mathbf{x},$$

where $\mathbf{H}$ is the *convolution matrix* built from the impulse response. For an impulse response of length M and an input of length L, the full convolution output has length $L + M - 1$, so $\mathbf{H}$ has dimensions $(L + M - 1) \times L$. Each column of $\mathbf{H}$ contains a copy of h shifted down by one row, giving the matrix a banded structure with constant values along each diagonal — a so-called *Toeplitz* matrix.[10]

Writing convolution in matrix form is more than a notational convenience: it allows the entire toolbox of linear algebra — inverses, least-squares solutions, eigenvalue and singular value analysis — to be applied to filtering problems such as deconvolution (undoing the blur of a measurement system) and channel equalization in communications.

[10] If the convolution is instead *circular* (periodic), the shifted copies of h wrap around and $\mathbf{H}$ becomes a *circulant* matrix. Circulant matrices are diagonalized by the discrete Fourier transform, which is why fast FFT-based convolution is possible — convolution in time is reduced to element-wise multiplication in frequency. In MATLAB, $\mathrm{conv}(h, x)$ computes the convolution directly and $\mathrm{convmtx}(h, L)$ constructs the convolution matrix $\mathbf{H}$.

Example: FIR filtering as a matrix product

Let a simple FIR filter have impulse response $h = [1,\ 2]$ (so $M = 2$) and let the input be $x = [1,\ 2,\ 3]$ (so $L = 3$). The output has length $L + M - 1 = 4$, and the convolution matrix is assembled by placing shifted copies of h in each column:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix},$$

so that

$$\mathbf{y} = \mathbf{H}\mathbf{x} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \\ 4 \\ 7 \\ 6 \end{bmatrix}.$$

The same result follows from the convolution sum directly: $y[0] = h[0]x[0] = 1$; $y[1] = h[0]x[1] + h[1]x[0] = 2 + 2 = 4$; $y[2] = h[0]x[2] + h[1]x[1] = 3 + 4 = 7$; and $y[3] = h[1]x[2] = 6$. Note the Toeplitz structure of $\mathbf{H}$: every diagonal is constant, reflecting the time-invariance of the filter.

Least-Squares Solutions and the Pseudoinverse

Laboratory measurements routinely produce *overdetermined* systems: more equations than unknowns, as when a straight line must be fit through many noisy data points. In that case $\mathbf{A}$ is a tall $m \times n$ matrix with $m > n$, no exact solution to $\mathbf{A}\mathbf{x} = \mathbf{b}$ exists in general, and the best that can be done is to find the $\hat{\mathbf{x}}$ that minimizes the squared magnitude of the residual error, $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2$. Setting the gradient of this quantity to zero yields the *normal equations*,

$$\mathbf{A}^\mathsf{T}\mathbf{A}\,\hat{\mathbf{x}} = \mathbf{A}^\mathsf{T}\mathbf{b} \quad \Rightarrow \quad \hat{\mathbf{x}} = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\,\mathbf{A}^\mathsf{T}\mathbf{b},$$

where the matrix $\mathbf{A}^+ = (\mathbf{A}^\mathsf{T}\mathbf{A})^{-1}\mathbf{A}^\mathsf{T}$ is known as the (Moore-Penrose) *pseudoinverse* of $\mathbf{A}$; it generalizes the matrix inverse to non-square matrices and reduces to $\mathbf{A}^{-1}$ when $\mathbf{A}$ is square and invertible.[11]

[11] Numerically robust implementations compute the pseudoinverse from the SVD as $\mathbf{A}^+ = \mathbf{V}\boldsymbol{\Sigma}^+\mathbf{U}^\mathsf{T}$, where $\boldsymbol{\Sigma}^+$ inverts each nonzero singular value. In MATLAB, the backslash operator $\mathbf{A}\backslash\mathbf{b}$ automatically returns the least-squares solution for overdetermined systems, and $\mathrm{pinv}(\mathbf{A})$ computes the pseudoinverse explicitly.

Example: Least-squares line fit

Fit a line $y = mx + c$ through the measured points $(1,1)$, $(2,2)$, and $(3,2)$. Each point contributes one equation $mx_i + c = y_i$, giving the overdetermined system

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 2 & 1 \\ 3 & 1 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} m \\ c \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}}_{\mathbf{b}}.$$

Forming the normal equations,

$$\mathbf{A}^\mathsf{T}\mathbf{A} = \begin{bmatrix} 14 & 6 \\ 6 & 3 \end{bmatrix}, \qquad \mathbf{A}^\mathsf{T}\mathbf{b} = \begin{bmatrix} 11 \\ 5 \end{bmatrix},$$

and applying the analytic 2×2 inverse with $\det(\mathbf{A}^\mathsf{T}\mathbf{A}) = 42 - 36 = 6$,

$$\hat{\mathbf{x}} = \begin{bmatrix} m \\ c \end{bmatrix} = \frac{1}{6}\begin{bmatrix} 3 & -6 \\ -6 & 14 \end{bmatrix}\begin{bmatrix} 11 \\ 5 \end{bmatrix} = \frac{1}{6}\begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 1/2 \\ 2/3 \end{bmatrix}.$$

The best-fit line is therefore $y = \frac{1}{2}x + \frac{2}{3}$, with residuals $(+\frac{1}{6}, -\frac{1}{3}, +\frac{1}{6})$ at the three points — no line passes exactly through all three, but this one minimizes the sum of the squared errors.

Exercises

Problem 1: Compute both the eigenvalues and the singular values of the matrix $\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & -2 \end{bmatrix}$. Why do the two sets differ, and what property of singular values does this illustrate?

Problem 2: A dataset consists of the four measurements $(3,1)$, $(-3,-1)$, $(1,3)$, and $(-1,-3)$. Compute the covariance matrix, find the principal components and their variances, and determine what fraction of the total variance is captured by the first principal component.

Problem 3: A first-difference FIR filter has impulse response $h = [1,\ -1]$. Construct the convolution matrix $\mathbf{H}$ for an input of length $L = 4$ and use it to compute the output for $x = [1,\ 3,\ 6,\ 10]$. Interpret the result.

Problem 4: Using the normal equations, find the least-squares line $y = mx + c$ through the points $(0,1)$, $(1,3)$, and $(2,4)$, and compute the residual at each point.

Section 42: Linear Algebra Fundamentals — Solutions to Exercises

This companion document provides complete, step-by-step solutions to all exercises in Section 42. Problems are grouped by the subsection in which they appear.

Section 42.1: Vectors

Problem 1: Unit vector

Given a vector $\boldsymbol{v} = \begin{bmatrix} 4 \\ 3 \end{bmatrix}$*, find the unit vector* $\boldsymbol{u}$ *in the direction of* $\boldsymbol{v}$*.*

The unit vector is obtained by dividing $\mathbf{v}$ by its magnitude. First compute the 2-norm,

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2} = \sqrt{4^2 + 3^2} = \sqrt{25} = 5.$$

Then divide each component by the magnitude:

$$\mathbf{u} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2} = \frac{1}{5}\begin{bmatrix} 4 \\ 3 \end{bmatrix} = \begin{bmatrix} 4/5 \\ 3/5 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.6 \end{bmatrix}.$$

As a check, $\|\mathbf{u}\|_2 = \sqrt{(4/5)^2 + (3/5)^2} = \sqrt{16/25 + 9/25} = \sqrt{25/25} = 1$, confirming unit magnitude.

Problem 2: Inner product

Calculate the inner product of vectors $\boldsymbol{a} = \begin{bmatrix} 1 \\ 2 \\ -1 \end{bmatrix}$ *and* $\boldsymbol{b} = \begin{bmatrix} -2 \\ 0 \\ 3 \end{bmatrix}$*.*

For real vectors, the inner product is the element-wise multiply-accumulate

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^{3} a_i b_i = (1)(-2) + (2)(0) + (-1)(3) = -5.$$

Since the inner product is nonzero, $\mathbf{a}$ and $\mathbf{b}$ are not orthogonal; the negative sign indicates the angle between them exceeds $90°$. Indeed, $\cos(\theta) = \frac{\mathbf{a}\cdot\mathbf{b}}{\|\mathbf{a}\|\|\mathbf{b}\|} = \frac{-5}{\sqrt{6}\sqrt{13}} \approx -0.566$, so $\theta \approx 124.5°$.

Problem 3: Complex orthogonal vectors

Consider two complex orthogonal vectors $\boldsymbol{v} = \begin{bmatrix} 3+4\mathrm{i} \\ a \end{bmatrix}$ *and* $\boldsymbol{u} = \frac{1}{2}\begin{bmatrix} 1+\mathrm{i} \\ b \end{bmatrix}$. *Solve for all possible values of* a *and* b, *assuming that* $\boldsymbol{u}$ *is a unit vector and* $b \in \mathbb{R}$.

Step 1: Apply the unit-vector condition to find b**.** Normalization of a complex vector requires $\|\mathbf{u}\| = \sqrt{\mathbf{u}^\dagger \mathbf{u}} = 1$, i.e., the sum of the squared moduli of the elements equals one:

$$\|\mathbf{u}\|^2 = \tfrac{1}{4}\left(|1+\mathrm{i}|^2 + |b|^2\right) = \tfrac{1}{4}\left(2 + b^2\right) = 1.$$

Solving for b,

$$2 + b^2 = 4 \quad \Rightarrow \quad b^2 = 2 \quad \Rightarrow \quad b = \pm\sqrt{2},$$

where both roots are admissible since $b \in \mathbb{R}$.

Step 2: Apply the orthogonality condition to find a**.** Two complex vectors are orthogonal if their inner product (using the Hermitian adjoint) vanishes:

$$\mathbf{u}^\dagger \cdot \mathbf{v} = \tfrac{1}{2}\begin{bmatrix} (1+\mathrm{i})^* & b^* \end{bmatrix}\begin{bmatrix} 3+4\mathrm{i} \\ a \end{bmatrix} = \tfrac{1}{2}\left[(1-\mathrm{i})(3+4\mathrm{i}) + b\,a\right] = 0,$$

where $b^* = b$ because b is real. Expanding the complex product,

$$(1-\mathrm{i})(3+4\mathrm{i}) = 3 + 4\mathrm{i} - 3\mathrm{i} - 4\mathrm{i}^2 = 3 + \mathrm{i} + 4 = 7 + \mathrm{i},$$

so the orthogonality condition becomes

$$7 + \mathrm{i} + b\,a = 0 \quad \Rightarrow \quad a = -\frac{7+\mathrm{i}}{b}.$$

Step 3: Combine. Substituting the two values of b:

$$b = +\sqrt{2}: \quad a = -\frac{7+\mathrm{i}}{\sqrt{2}} = -\frac{\sqrt{2}}{2}(7+\mathrm{i}),$$
$$b = -\sqrt{2}: \quad a = +\frac{7+\mathrm{i}}{\sqrt{2}} = +\frac{\sqrt{2}}{2}(7+\mathrm{i}).$$

Thus there are exactly two solution pairs, $(a, b) = \left(\mp\frac{\sqrt{2}}{2}(7+\mathrm{i}),\ \pm\sqrt{2}\right)$. As a check for $b = \sqrt{2}$: $\mathbf{u}^\dagger\mathbf{v} = \frac{1}{2}[(7+\mathrm{i}) + \sqrt{2}\cdot(-(7+\mathrm{i})/\sqrt{2})] = \frac{1}{2}[(7+\mathrm{i}) - (7+\mathrm{i})] = 0$. ✓

Section 42.2: Matrices

Problem 1: Matrix products

Given the matrices $\boldsymbol{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ *and* $\boldsymbol{B} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$*, compute both matrix products* $\boldsymbol{AB}$ *and* $\boldsymbol{BA}$*, and thereby confirm that matrix multiplication is not commutative. In addition, compute the Hadamard product* $\boldsymbol{A} \odot \boldsymbol{B}$ *and the tensor product* $\boldsymbol{A} \otimes \boldsymbol{B}$*.*

Standard (MAC) products. Each element of the product is a row-column multiply-accumulate:

$$\begin{aligned} \mathbf{AB} &= \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \\ &= \begin{bmatrix} (1)(0)+(2)(1) & (1)(1)+(2)(0) \\ (3)(0)+(4)(1) & (3)(1)+(4)(0) \end{bmatrix} \\ &= \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix}, \end{aligned}$$

$$\begin{aligned} \mathbf{BA} &= \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \\ &= \begin{bmatrix} (0)(1)+(1)(3) & (0)(2)+(1)(4) \\ (1)(1)+(0)(3) & (1)(2)+(0)(4) \end{bmatrix} \\ &= \begin{bmatrix} 3 & 4 \\ 1 & 2 \end{bmatrix}. \end{aligned}$$

Since $\mathbf{AB} \neq \mathbf{BA}$, matrix multiplication is confirmed to be non-commutative. The structure of the disagreement is instructive: right-multiplication by the permutation matrix $\mathbf{B}$ swaps the *columns* of $\mathbf{A}$, whereas left-multiplication swaps its *rows*.

Hadamard product. Multiplying element-by-element,

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} (1)(0) & (2)(1) \\ (3)(1) & (4)(0) \end{bmatrix} = \begin{bmatrix} 0 & 2 \\ 3 & 0 \end{bmatrix}.$$

Tensor (Kronecker) product. Each element of $\mathbf{A}$ scales an entire copy of $\mathbf{B}$, producing a 4×4 block matrix:

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} 1\mathbf{B} & 2\mathbf{B} \\ 3\mathbf{B} & 4\mathbf{B} \end{bmatrix} = \left[\begin{array}{cc|cc} 0 & 1 & 0 & 2 \\ 1 & 0 & 2 & 0 \\ \hline 3 & 0 & 4 & 0 \\ 0 & 3 & 0 & 4 \end{array}\right].$$

Problem 2: Determinant and inverse

Compute the determinant and the inverse of the matrix $\boldsymbol{A} = \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix}$, *and verify that* $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}$.

The determinant is

$$\det(\mathbf{A}) = a_{11}a_{22} - a_{12}a_{21} = (2)(3) - (1)(5) = 6 - 5 = 1.$$

Since $\det(\mathbf{A}) = 1 \neq 0$, the matrix is invertible. Applying the analytic 2×2 inverse formula (swap the diagonal elements, negate the off-diagonal elements, divide by the determinant):

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \begin{bmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{bmatrix} = \frac{1}{1} \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix} = \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix}.$$

Verification by direct multiplication:

$$\begin{aligned} \mathbf{A}^{-1}\mathbf{A} &= \begin{bmatrix} 3 & -1 \\ -5 & 2 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 5 & 3 \end{bmatrix} \\ &= \begin{bmatrix} (3)(2) + (-1)(5) & (3)(1) + (-1)(3) \\ (-5)(2) + (2)(5) & (-5)(1) + (2)(3) \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \mathbf{I}. \checkmark \end{aligned}$$

Problem 3: Linear system two ways

Solve the linear system $\begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \end{bmatrix}$ *twice: once using the analytic* 2×2 *matrix inverse, and once using Cramer's rule. Confirm that both methods agree.*

In both approaches the determinant of the coefficient matrix is needed:

$$\det(\mathbf{A}) = (3)(2) - (-1)(-1) = 6 - 1 = 5.$$

Method 1: matrix inverse.

$$\mathbf{A}^{-1} = \frac{1}{5} \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix},$$

so that

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b} = \frac{1}{5}\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}\begin{bmatrix} 5 \\ 0 \end{bmatrix} = \frac{1}{5}\begin{bmatrix} (2)(5)+(1)(0) \\ (1)(5)+(3)(0) \end{bmatrix}$$
$$= \frac{1}{5}\begin{bmatrix} 10 \\ 5 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Method 2: Cramer's rule. Each unknown is a ratio of determinants, with the i-th column of $\mathbf{A}$ replaced by $\mathbf{b}$ in the numerator:

$$x_1 = \frac{|\mathbf{A}_1|}{|\mathbf{A}|} = \frac{\begin{vmatrix} 5 & -1 \\ 0 & 2 \end{vmatrix}}{5} = \frac{(5)(2)-(-1)(0)}{5} = \frac{10}{5} = 2,$$

$$x_2 = \frac{|\mathbf{A}_2|}{|\mathbf{A}|} = \frac{\begin{vmatrix} 3 & 5 \\ -1 & 0 \end{vmatrix}}{5} = \frac{(3)(0)-(5)(-1)}{5} = \frac{5}{5} = 1.$$

Both methods agree: $x_1 = 2$, $x_2 = 1$. Substituting back as a final check: $3(2) - 1(1) = 5$ ✓ and $-1(2) + 2(1) = 0$ ✓.

Problem 4: Eigenvalues and eigenvectors

Find the eigenvalues and unit-normalized eigenvectors of $\boldsymbol{A} = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}$. *Verify your eigenvalues by checking them against the trace and determinant of* $\boldsymbol{A}$.

Step 1: Characteristic equation.

$$\det(\mathbf{A} - \lambda\mathbf{I}) = \begin{vmatrix} 4-\lambda & 1 \\ 2 & 3-\lambda \end{vmatrix} = (4-\lambda)(3-\lambda) - 2$$
$$= \lambda^2 - 7\lambda + 10 = 0.$$

Factoring,

$$\lambda^2 - 7\lambda + 10 = (\lambda - 2)(\lambda - 5) = 0 \quad \Rightarrow \quad \lambda_1 = 2, \quad \lambda_2 = 5.$$

Step 2: Eigenvector for $\lambda_1 = 2$. Solve $(\mathbf{A} - 2\mathbf{I})\mathbf{v}_1 = \mathbf{o}$:

$$\begin{bmatrix} 2 & 1 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \Rightarrow \quad 2v_x + v_y = 0 \quad \Rightarrow \quad v_y = -2v_x.$$

Taking $v_x = 1$ gives $[1, -2]^\intercal$ with magnitude $\sqrt{1+4} = \sqrt{5}$, so

$$\mathbf{v}_1 = \frac{1}{\sqrt{5}}\begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

Step 3: Eigenvector for $\lambda_2 = 5$. Solve $(\mathbf{A} - 5\mathbf{I})\mathbf{v}_2 = \mathbf{0}$:

$$\begin{bmatrix} -1 & 1 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \Rightarrow \quad -v_x + v_y = 0 \quad \Rightarrow \quad v_y = v_x,$$

so with normalization

$$\mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Direct verification: $\mathbf{A}\mathbf{v}_2 = \frac{1}{\sqrt{2}}[4+1,\ 2+3]^\intercal = \frac{1}{\sqrt{2}}[5, 5]^\intercal = 5\mathbf{v}_2$. ✓

Step 4: Trace and determinant checks.

$$\lambda_1 + \lambda_2 = 2 + 5 = 7 = 4 + 3 = \mathrm{tr}(\mathbf{A}),$$
$$\lambda_1\lambda_2 = (2)(5) = 10 = (4)(3) - (1)(2) = \det(\mathbf{A}). \checkmark$$

Note that since $\mathbf{A}$ is *not* symmetric, the eigenvectors need not be orthogonal — and indeed $\mathbf{v}_1 \cdot \mathbf{v}_2 = \frac{1}{\sqrt{10}}(1 - 2) \neq 0$.

Section 42.3: Advanced Topics

Problem 1: Eigenvalues vs. singular values

Compute both the eigenvalues and the singular values of the matrix $\boldsymbol{A} = \begin{bmatrix} 1 & 0 \\ 0 & -2 \end{bmatrix}$. *Why do the two sets differ, and what property of singular values does this illustrate?*

Eigenvalues. The matrix is diagonal, so its eigenvalues are simply the diagonal entries:

$$\det(\mathbf{A} - \lambda\mathbf{I}) = (1 - \lambda)(-2 - \lambda) = 0 \quad \Rightarrow \quad \lambda_1 = 1, \quad \lambda_2 = -2.$$

Singular values. Form the symmetric matrix $\mathbf{A}^\intercal\mathbf{A}$ and take the square roots of its eigenvalues:

$$\mathbf{A}^\intercal\mathbf{A} = \begin{bmatrix} 1 & 0 \\ 0 & -2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & -2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix} \quad \Rightarrow \quad \lambda(\mathbf{A}^\intercal\mathbf{A}) = \{1, 4\},$$

so that, sorted in descending order,

$$\sigma_1 = \sqrt{4} = 2, \qquad \sigma_2 = \sqrt{1} = 1.$$

Interpretation. The sets differ because the eigenvalue -2 appears as the singular value $+2$: singular values are *always real*

and non-negative by definition, since they derive from the (positive semi-definite) matrix $\mathbf{A}^\mathsf{T}\mathbf{A}$. For this diagonal (hence symmetric) matrix, $\sigma_i = |\lambda_i|$. Geometrically, the eigenvalue -2 describes a stretch by a factor of 2 combined with a reflection (sign flip) along the second axis; the singular value records only the *magnitude* of the amplification, $|-2| = 2$, consistent with the unit-circle-to-ellipse picture in which semi-axis lengths cannot be negative. The eigenvalue set need not even be real for a general matrix, whereas the singular values always are.

Problem 2: PCA of a four-point dataset

A dataset consists of the four measurements $(3,1)$, $(-3,-1)$, $(1,3)$, *and* $(-1,-3)$. *Compute the covariance matrix, find the principal components and their variances, and determine what fraction of the total variance is captured by the first principal component.*

Step 1: Mean-center the data. The mean is

$$\boldsymbol{\mu} = \tfrac{1}{4}\big[(3,1)+(-3,-1)+(1,3)+(-1,-3)\big] = (0,0),$$

so the data are already centered and the deviation vectors are the measurements themselves:

$$\mathbf{d}_1 = \begin{bmatrix}3\\1\end{bmatrix}, \quad \mathbf{d}_2 = \begin{bmatrix}-3\\-1\end{bmatrix}, \quad \mathbf{d}_3 = \begin{bmatrix}1\\3\end{bmatrix}, \quad \mathbf{d}_4 = \begin{bmatrix}-1\\-3\end{bmatrix}.$$

Step 2: Covariance matrix. With $N-1=3$, sum the outer products:

$$\mathbf{d}_1\mathbf{d}_1^\mathsf{T} = \mathbf{d}_2\mathbf{d}_2^\mathsf{T} = \begin{bmatrix}9 & 3\\3 & 1\end{bmatrix}, \qquad \mathbf{d}_3\mathbf{d}_3^\mathsf{T} = \mathbf{d}_4\mathbf{d}_4^\mathsf{T} = \begin{bmatrix}1 & 3\\3 & 9\end{bmatrix},$$

$$\mathbf{C} = \frac{1}{3}\sum_{i=1}^{4}\mathbf{d}_i\mathbf{d}_i^\mathsf{T} = \frac{1}{3}\begin{bmatrix}20 & 12\\12 & 20\end{bmatrix},$$

where, for example, the top-left entry accumulates $C_{11} = \frac{1}{3}(9+9+1+1) = \frac{20}{3}$.

Step 3: Principal components and variances. The covariance matrix is symmetric with equal diagonal entries, so (as in the eigenvalue examples of Section 42.2) its unit eigenvectors are

$$\mathbf{w}_1 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\1\end{bmatrix}, \qquad \mathbf{w}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix}1\\-1\end{bmatrix},$$

with eigenvalues obtained by applying $\mathbf{C}$ along each direction:

$$\lambda_1 = \frac{20+12}{3} = \frac{32}{3}, \qquad \lambda_2 = \frac{20-12}{3} = \frac{8}{3}.$$

The first principal component points along the diagonal $[1,1]^\intercal$ direction with variance $32/3$; the second is orthogonal to it with variance $8/3$.

Step 4: Variance fraction.

$$\frac{\lambda_1}{\lambda_1+\lambda_2}=\frac{32/3}{32/3+8/3}=\frac{32}{40}=80\%.$$

The first principal component captures 80% of the total variance; projecting each 2D measurement onto $\mathbf{w}_1$ would preserve most, though not all, of the structure in the data.

Problem 3: First-difference convolution matrix

A first-difference FIR filter has impulse response $h=[1,-1]$. Construct the convolution matrix $\boldsymbol{H}$ for an input of length $L=4$ and use it to compute the output for $\boldsymbol{x}=[1,3,6,10]$. Interpret the result.

Step 1: Dimensions. With impulse-response length $M=2$ and input length $L=4$, the full convolution output has length $L+M-1=5$, so $\mathbf{H}$ is a 5×4 matrix.

Step 2: Assemble H. Each column contains a copy of $h=[1,-1]$ shifted down by one row (Toeplitz structure):

$$\mathbf{H}=\begin{bmatrix}1&0&0&0\\-1&1&0&0\\0&-1&1&0\\0&0&-1&1\\0&0&0&-1\end{bmatrix}.$$

Step 3: Compute the output.

$$\mathbf{y}=\mathbf{H}\mathbf{x}=\begin{bmatrix}1&0&0&0\\-1&1&0&0\\0&-1&1&0\\0&0&-1&1\\0&0&0&-1\end{bmatrix}\begin{bmatrix}1\\3\\6\\10\end{bmatrix}=\begin{bmatrix}1\\3-1\\6-3\\10-6\\0-10\end{bmatrix}=\begin{bmatrix}1\\2\\3\\4\\-10\end{bmatrix}.$$

Step 4: Interpretation. The interior samples compute successive differences $y[n]=x[n]-x[n-1]$ — the discrete analogue of a derivative. The input $[1,3,6,10]$ is a sequence of cumulative (running) sums — the triangular numbers, whose increments are $1,2,3,4$ — and differentiating it recovers exactly that underlying increment sequence $[1,2,3,4]$: differencing undoes summation, just as differentiation undoes integration. The first

sample ($y[0] = x[0] = 1$, which here coincides with the first increment) and the last sample ($y[4] = -x[3] = -10$) are boundary effects of the full convolution, arising from the implicit zeros assumed outside the input record. In circuit terms, this filter is a crude discrete-time differentiator (a high-pass operation), and its output magnitude is largest where the input changes most abruptly — here, at the artificial step back to zero at the end of the record.

Problem 4: Least-squares line fit

Using the normal equations, find the least-squares line $y = mx + c$ through the points $(0,1)$, $(1,3)$, and $(2,4)$, and compute the residual at each point.

Step 1: Set up the overdetermined system. Each point contributes one equation $mx_i + c = y_i$:

$$\underbrace{\begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 1 \end{bmatrix}}_{\mathbf{A}} \underbrace{\begin{bmatrix} m \\ c \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} 1 \\ 3 \\ 4 \end{bmatrix}}_{\mathbf{b}}.$$

Three equations in two unknowns: no exact solution exists (the three points are not collinear), so we seek the least-squares solution.

Step 2: Form the normal equations.

$$\mathbf{A}^\intercal\mathbf{A} = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 \\ 1 & 1 \\ 2 & 1 \end{bmatrix} = \begin{bmatrix} 0+1+4 & 0+1+2 \\ 0+1+2 & 1+1+1 \end{bmatrix} = \begin{bmatrix} 5 & 3 \\ 3 & 3 \end{bmatrix},$$

$$\mathbf{A}^\intercal\mathbf{b} = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 0+3+8 \\ 1+3+4 \end{bmatrix} = \begin{bmatrix} 11 \\ 8 \end{bmatrix}.$$

Step 3: Solve. With $\det(\mathbf{A}^\intercal\mathbf{A}) = (5)(3) - (3)(3) = 15 - 9 = 6$, the analytic inverse gives

$$\hat{\mathbf{x}} = \begin{bmatrix} m \\ c \end{bmatrix} = (\mathbf{A}^\intercal\mathbf{A})^{-1}\mathbf{A}^\intercal\mathbf{b} = \frac{1}{6}\begin{bmatrix} 3 & -3 \\ -3 & 5 \end{bmatrix}\begin{bmatrix} 11 \\ 8 \end{bmatrix} = \frac{1}{6}\begin{bmatrix} 33-24 \\ -33+40 \end{bmatrix}$$

$$= \frac{1}{6}\begin{bmatrix} 9 \\ 7 \end{bmatrix} = \begin{bmatrix} 3/2 \\ 7/6 \end{bmatrix}.$$

The best-fit line is therefore

$$y = \frac{3}{2}x + \frac{7}{6}.$$

Step 4: Residuals. Evaluating the fitted line at each point, $r_i = (mx_i + c) - y_i$:

$$\begin{aligned} x = 0: &\quad r_1 = \tfrac{7}{6} - 1 = +\tfrac{1}{6}, \\ x = 1: &\quad r_2 = \tfrac{3}{2} + \tfrac{7}{6} - 3 = \tfrac{16}{6} - 3 = -\tfrac{1}{3}, \\ x = 2: &\quad r_3 = 3 + \tfrac{7}{6} - 4 = +\tfrac{1}{6}. \end{aligned}$$

The residual vector is $\mathbf{r} = \left[+\tfrac{1}{6}, -\tfrac{1}{3}, +\tfrac{1}{6}\right]^{\mathsf{T}}$: the line passes slightly above the outer points and below the middle one. Two checks confirm the least-squares property: the residuals sum to zero (guaranteed by the column of ones in $\mathbf{A}$), and $\mathbf{r}$ is orthogonal to both columns of $\mathbf{A}$, e.g., $(0)(\tfrac{1}{6}) + (1)(-\tfrac{1}{3}) + (2)(\tfrac{1}{6}) = 0$. ✓ No line passes through all three points, but this one minimizes $\|\mathbf{r}\|_2^2 = \tfrac{1}{36} + \tfrac{1}{9} + \tfrac{1}{36} = \tfrac{1}{6}$.